Final Lab Report: Digital Fuel Pump System

ECEN 2233 30770 – Fundamentals of Digital Logic Design

Written By: Shaheen, Daniel

I. Introduction

During this lab we designed a digital system which simulates the operation of a fuel pump, providing all the well-known functionalities such as calculating the required fuel to fill a tank, determining the total cost of the fuel, and displaying the final cost incrementally on a seven-segment display. This design shows case key digital design concepts, including Two's Complement, subtractors and adders, counters, finite state machines (FSMs), and seven-segment displays. We brought these components together to create a realistic and interactive experience that mirrors the workings of your everyday fuel pump.

This project serves as the practical application of theoretical digital design principles while also enhancing the users experience through visual feedback. Using a modular approach, you can help ensure that each subsystem—calculation, control, and display—functions within the top-level integration work seamlessly. The digital fuel pump system is designed to not only demonstrate technical proficiency but also engage users with an animated, educational interface.

II. Fuel Needed Calculator

The fuel needed calculator determines the exact amount of fuel required to fill a gas tank to its maximum capacity. This is achieved by performing a subtraction operation using Two's Complement arithmetic, a fundamental concept in digital systems. The logic is implemented in a straightforward, yet effective way: the amount of fuel needed is calculated as the difference between the tank's total capacity and the fuel currently in the tank.

By employing Two's Complement subtraction, the system ensures accuracy and reliability, even when dealing with binary arithmetic. This module is an essential component of the fuel pump system, providing the foundation for cost calculations while also ensuring the rest of the simulation operates based on precise data.

III. Cost Calculator

The cost calculator determines the total cost required to fill a given gas tank with the calculated fuel needed. It uses a simple but also vital arithmetic formula: the total cost is obtained by multiplying the required fuel quantity with the price per liter. This module integrates with the fuel-needed calculator by taking its output as one of its inputs.

The implementation involves a multiplication module capable of handling binary arithmetic efficiently. This ensures that the fuel system delivers accurate results, regardless of the range of input values. By automating the cost calculation, this module eliminates the need for manual computations, aligning with the goal of providing an intuitive and user-friendly experience.

IV. FSM Display Controller

The FSM (Finite State Machine) Display Controller adds an engaging, user-friendly dimension to the fuel pump system by animating the display of the total cost. Instead of instantly presenting the final amount after calculation, the system gradually counts from zero to the total cost, simulating the real-time operation of a fuel pump display. This interactive dynamic feedback offers a more realistic experience for users.

The FSM governs this behavior by controlling the seven-segment display, this is to ensure that it updates systematically and accurately. As the controller orchestrates the gradual incrementing of the display, it also emphasizes the practical application of FSMs in real-world systems. This feature not only enhances the user experience, but it also demonstrates the efficient use of digital design principles to achieve smooth and precise control over hardware components.

V. Top-Level Integration

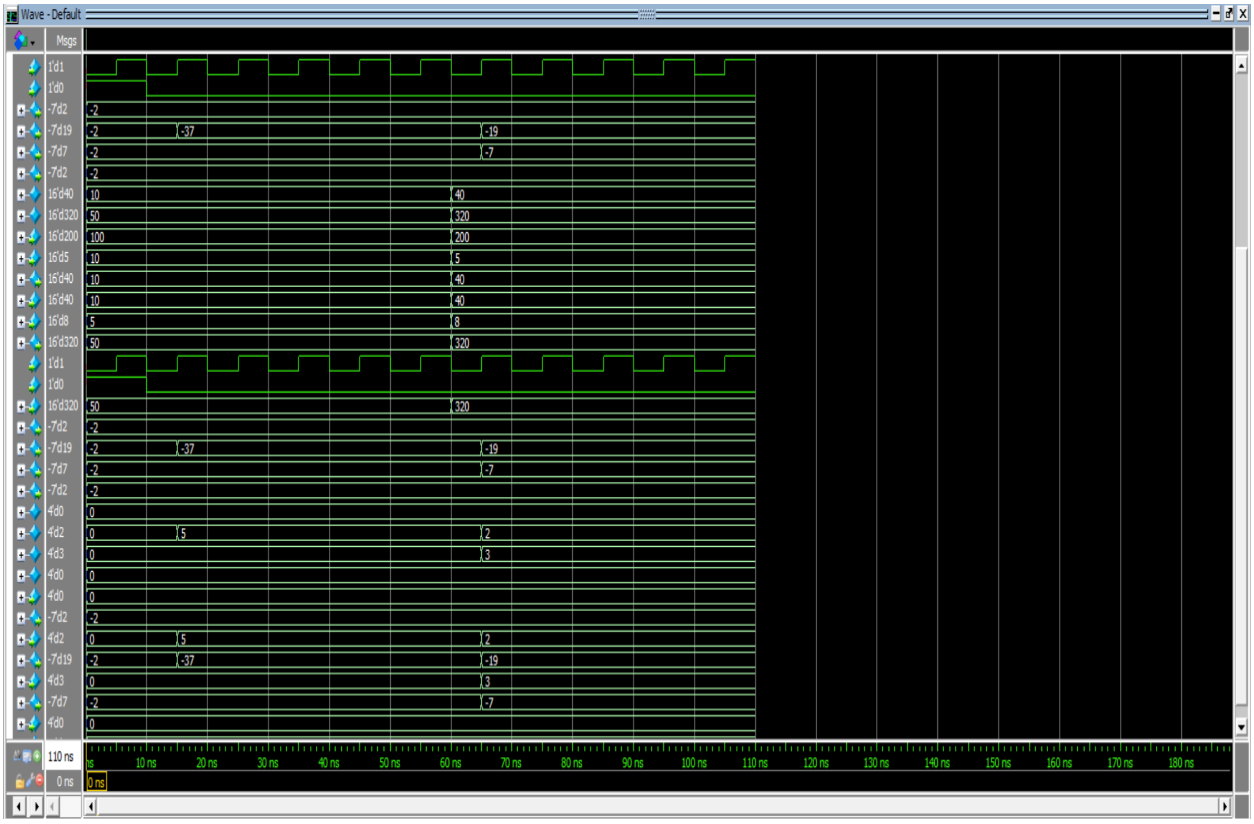
The top-level integration module connects all of the individual components of the fuel pump system, overall making it into a cohesive whole. It combines all given inputs such as the current fuel level, tank capacity, and fuel price per liter to compute outputs like the amount of fuel needed, the total cost, and the animated display values for a seven-segment display. This integration helps to ensure all modules work together hand in hand, transforming individual functionalities into a unified fuel pump simulation.

This module is used to handle the coordination among the fuel-needed calculator, cost calculator, FSM display controller, and seven-segment display outputs. It signifies the importance of modular design, where each subsystem functions independently but still contributes collectively to the overall, main, operation. By managing the overall flow of data across the modules, the top-level integration guarantees reliability as well as accuracy throughout the entire system.

VI. Conclusion

In summary, this digital system is made to successfully simulate the basic operations of a fuel pump while also reinforcing the crucial digital design principles. Through the use of modular design and integration, the system can demonstrate the practicality and efficiency of all components like Two's Complement, FSMs, and seven-segment displays. The interactive features, such as the animated cost display, add a layer of realism and user engagement to the project, overall enhancing the experience.

This project showcases the real-world applications and the theoretical concepts of a fuel pump, making it an excellent educational tool for understanding complex digital design systems. By bringing together the technical knowledge, with the given practical implementation, the system can serve as a robust example of how digital technologies can create functional, intuitive, and engaging solutions.




```
module cost_calculator (
    input signed [15:0] fuel_needed,
    input signed [15:0] cost_per_liter,
    output signed [15:0] total_cost
);
    assign total_cost = fuel_needed * cost_per_liter;
endmodule
```

```
module fuel_needed_calculator (
    input signed [15:0] distance,
    input signed [15:0] efficiency,
    output signed [15:0] fuel_needed
);
    assign fuel_needed = distance / efficiency;
endmodule
```

```
module seven_segment_display (
    input [3:0] digit,
    output reg [6:0] segments
);
    always @(*) begin
        case (digit)
            4'b0000: segments = 7'b1111110; // 0
            4'b0001: segments = 7'b0110000; // 1
            4'b0010: segments = 7'b1101101; // 2
            4'b0011: segments = 7'b1111001; // 3
            4'b0100: segments = 7'b0110011; // 4
            4'b0101: segments = 7'b1011011; // 5
            4'b0110: segments = 7'b1011111; // 6
            4'b0111: segments = 7'b1110000; // 7
            4'b1000: segments = 7'b1111111; // 8
            4'b1001: segments = 7'b1111011; // 9
            default: segments = 7'b0000000; // Off
        endcase
    end
endmodule
```

```
module display_fsm (
    input logic clk,
    input logic reset,
    input logic [15:0] total_cost,
    output reg [6:0] segment_1,
    output reg [6:0] segment_2,
    output reg [6:0] segment_3,
    output reg [6:0] segment_4
);
    reg [3:0] digit_1, digit_2, digit_3, digit_4;

    always_ff @(posedge clk or posedge reset) begin
        if (reset) begin
            digit_1 <= 0;
            digit_2 <= 0;
            digit_3 <= 0;
            digit_4 <= 0;
        end else begin
            digit_1 <= total_cost % 10;
            digit_2 <= (total_cost / 10) % 10;
            digit_3 <= (total_cost / 100) % 10;
            digit_4 <= (total_cost / 1000) % 10;
        end
    end

    seven_segment_display display1 (.digit(digit_1), .segments(segment_1));
    seven_segment_display display2 (.digit(digit_2), .segments(segment_2));
    seven_segment_display display3 (.digit(digit_3), .segments(segment_3));
    seven_segment_display display4 (.digit(digit_4), .segments(segment_4));

endmodule
```

```
module top_level (
    input signed [15:0] distance,
    input signed [15:0] efficiency,
    input signed [15:0] cost_per_liter,
    input logic clk,
    input logic reset,
    output [6:0] segment_1,
    output [6:0] segment_2,
    output [6:0] segment_3,
    output [6:0] segment_4
);
    wire signed [15:0] fuel_needed;
    wire signed [15:0] total_cost;

    fuel_needed_calculator fuel_calc (
        .distance(distance),
        .efficiency(efficiency),
        .fuel_needed(fuel_needed)
    );

    cost_calculator cost_calc (
        .fuel_needed(fuel_needed),
        .cost_per_liter(cost_per_liter)
        .total_cost(total_cost)
    );

    display_fsm display (
        .clk(clk),
        .reset(reset),
        .total_cost(total_cost),
        .segment_1(segment_1),
        .segment_2(segment_2),
        .segment_3(segment_3),
        .segment_4(segment_4)
    );

endmodule
```

```
module testbench;

    reg signed [15:0] distance;
    reg signed [15:0] efficiency;
    reg signed [15:0] cost_per_liter;
    reg clk;
    reg reset;

    wire [6:0] segment_1, segment_2, segment_3, segment_4;

    top_level uut (
        .distance(distance),
        .efficiency(efficiency),
        .cost_per_liter(cost_per_liter),
        .clk(clk),
        .reset(reset),
        .segment_1(segment_1),
        .segment_2(segment_2),
        .segment_3(segment_3),
        .segment_4(segment_4)
    );

    always begin
        #5 clk = ~clk;
    end

    initial begin
        clk = 0;
        reset = 1;
        distance = 100;
        efficiency = 10;
        cost_per_liter = 5;

        #10 reset = 0;
        #50;

        distance = 200;
        efficiency = 5;
        cost_per_liter = 4;

        #50;
        $finish;
    end

endmodule
```